

```
```bash echo -e "translate English to  
French: The weather is nice today." |
transformers run --task text2text-
generation --model google-t5/t5-base -
-device 0 ```
```

[https://huggingface.co/docs/transformers/main/en/model\\_doc/t5#transformers.T5Config](https://huggingface.co/docs/transformers/main/en/model_doc/t5#transformers.T5Config)

## Detailed Documentation

---

```
class <span class="code-
inline">transformers.T5Configtransformers.T5Confighttps
```

---

class

`transformers.T5Configtransformers.T5Confighttps` ://github.com/huggingfac  
"vocab\_size", "val": " = 32128"}, {"name": "d\_model", "val": " = 512"}, {"name":  
"d\_kv", "val": " = 64"}, {"name": "d\_ff", "val": " = 2048"}, {"name": "num\_layers",  
"val": " = 6"}, {"name": "num\_decoder\_layers", "val": " = None"}, {"name":  
"num\_heads", "val": " = 8"}, {"name": "relative\_attention\_num\_buckets", "val": "  
= 32"}, {"name": "relative\_attention\_max\_distance", "val": " = 128"}, {"name":  
"dropout\_rate", "val": " = 0.1"}, {"name": "layer\_norm\_epsilon", "val": " = 1e-06"},  
{"name": "initializer\_factor", "val": " = 1.0"}, {"name": "feed\_forward\_proj", "val": "

```
= 'relu'}, {"name": "is_encoder_decoder", "val": " = True"}, {"name": "use_cache", "val": " = True"}, {"name": "pad_token_id", "val": " = 0"}, {"name": "eos_token_id", "val": " = 1"}, {"name": "classifier_dropout", "val": " = 0.0"}, {"name": "***kwargs", "val": ""}] - **vocab_size** (int, *optional*, defaults to 32128) --
```

Vocabulary size of the T5 model. Defines the number of different tokens that can be represented by the

`inputs_ids` passed when calling `[T5Model]` (`/docs/transformers/main/en/model_doc/t5# transformers.T5Model`) or `TFT5Model`.

- `**d_model**` ( `int`, \*optional\*, defaults to 512) --

Size of the encoder layers and the pooler layer.

- `**d_kv**` ( `int`, \*optional\*, defaults to 64) --

Size of the key, query, value projections per attention head. The `inner_dim` of the projection layer will

be defined as `num_heads * d_kv`.

- `**d_ff**` ( `int`, \*optional\*, defaults to 2048) --

Size of the intermediate feed forward layer in each `T5Block`.

---

`class` `<span class="code-`

`transformers.T5Tokenizer`

---

class

```
transformers.T5Tokenizer(
 vocab_file=None, eos_token="<eos>", unk_token="<unk>", pad_token="<pad>",
 extra_ids=100, additional_special_tokens=None, sp_model_kwargs=None,
 legacy=False, add_prefix_space=False, **kwargs) --
 vocab_file (str) --
```

[SentencePiece](https://github.com/google/sentencepiece) `file` (generally has a \*.spm\* extension) that

contains the vocabulary necessary to instantiate a tokenizer.

- `eos_token` (str, \*optional\*, defaults to "<eos>") --

The end of sequence token.

When building a sequence using special tokens, this is not the token that is used for the end of sequence.

The token used is the `sep_token`.

- `unk_token` (str, \*optional\*, defaults to "<unk>") --

The unknown token. A token that is not in the vocabulary cannot be converted to an ID and is set to be this

- `**pad_token**` ( `str`, \*optional\*, defaults to `""` ) --

---

## build\_inputs\_with\_special\_tokenstransformers.T5Tokenizer.build

---

build\_inputs\_with\_special\_tokenstransformers.T5Tokenizer.build\_inputs\_with\_spe  
 "token\_ids\_0", "val": ": list"}, {"name": "token\_ids\_1", "val": ":  
 typing.Optional[list[int]] = None"]- \*\*token\_ids\_0\*\* ( `list[int]` ) --

List of IDs to which the special tokens will be added.

- `**token_ids_1**` ( `list[int]`, \*optional\*) --

Optional second list of IDs for sequence pairs.0 `list[int]` List of [input IDs]  
 (../glossary#input-ids) with the appropriate special tokens.

Build model inputs from a sequence or a pair of sequence for sequence  
 classification tasks by concatenating and

adding special tokens. A sequence has the following format:

- single sequence: `X`
- pair of sequences: `A B`

---

## get\_special\_tokens\_masktransformers.T5Tokenizer.get\_special\_t

---

get\_special\_tokens\_masktransformers.T5Tokenizer.get\_special\_tokens\_maskhttp  
 "token\_ids\_0", "val": ": list"}, {"name": "token\_ids\_1", "val": ":

```
typing.Optional[list[int]] = None"}, {"name": "already_has_special_tokens", "val":
": bool = False"}]- **token_ids_0** (list[int]) --
```

- \*\*token\_ids\_1\*\* ( list[int] , \*optional\*) --

Optional second list of IDs for sequence pairs.

- \*\*already\_has\_special\_tokens\*\* ( bool , \*optional\*, defaults to False ) --

Whether or not the token list is already formatted with special tokens for the model.0 list[int] A list of integers in the range [0, 1]: 1 for a special token, 0 for a sequence token.

Retrieve sequence ids from a token list that has no special tokens added. This method is called when adding

special tokens using the tokenizer prepare\_for\_model method.

---

## create\_token\_type\_ids\_from\_sequences transformers.T5Tokenizer

---

```
create_token_type_ids_from_sequences transformers.T5Tokenizer.create_token_t
"token_ids_0", "val": ": list"}, {"name": "token_ids_1", "val": ":
typing.Optional[list[int]] = None"}]- **token_ids_0** (list[int]) --
```

- \*\*token\_ids\_1\*\* ( list[int] , \*optional\*) --

Optional second list of IDs for sequence pairs.0 list[int] List of zeros.

Create a mask from the two sequences passed to be used in a sequence-pair classification task. T5 does not make

use of token type ids, therefore a list of zeros is returned.

---

## save\_vocabularytransformers.T5Tokenizer.save\_vocabularyhttps

---

```
save_vocabularytransformers.T5Tokenizer.save_vocabularyhttps://github.com/hu
"save_directory", "val": ": str"}, {"name": "filename_prefix", "val": ":
typing.Optional[str] = None"]}]
```

---

class <span class="code-

inline">transformers.T5TokenizerFasttransformers.T5TokenizerFa

---

class

```
transformers.T5TokenizerFasttransformers.T5TokenizerFasthttps://github.
"vocab_file", "val": " = None"}, {"name": "tokenizer_file", "val": " = None"},
{"name": "eos_token", "val": " = ""}, {"name": "unk_token", "val": " = ""},
{"name": "pad_token", "val": " = ""}, {"name": "extra_ids", "val": " = 100"},
{"name": "additional_special_tokens", "val": " = None"}, {"name":
"add_prefix_space", "val": " = None"}, {"name": "**kwargs", "val": ""}]-
vocab_file (str) --
```

[SentencePiece](https://github.com/google/sentencepiece) `file` (generally has  
a \*.spm\* extension) that

contains the vocabulary necessary to instantiate a tokenizer.

- `**eos_token**` ( `str`, \*optional\*, defaults to `""` ) --

The end of sequence token.

When building a sequence using special tokens, this is not the token that is used for the end of sequence.

The token used is the `sep_token`.

- `**unk_token**` ( `str`, \*optional\*, defaults to `""` ) --

The unknown token. A token that is not in the vocabulary cannot be converted to an ID and is set to be this

- `**pad_token**` ( `str`, \*optional\*, defaults to `""` ) --

---

## `build_inputs_with_special_tokenstransformers.T5TokenizerFast.b`

---

`build_inputs_with_special_tokenstransformers.T5TokenizerFast.build_inputs_with`  
"token\_ids\_0", "val": ": list"}, {"name": "token\_ids\_1", "val": ":  
typing.Optional[list[int]] = None"]- `**token_ids_0**` ( `list[int]` ) --

List of IDs to which the special tokens will be added.

- `**token_ids_1**` ( `list[int]`, \*optional\*) --

Optional second list of IDs for sequence pairs.0 `list[int]` List of [input IDs]  
(../glossary#input-ids) with the appropriate special tokens.

Build model inputs from a sequence or a pair of sequence for sequence classification tasks by concatenating and

adding special tokens. A sequence has the following format:

- single sequence: `X`
- pair of sequences: `A B`

---

## `create_token_type_ids_from_sequences``transformers.T5Tokenizer`

---

```
create_token_type_ids_from_sequences.T5TokenizerFast.create_token_type_ids_from_sequences(
 "token_ids_0", "val": "list", {"name": "token_ids_1", "val": "list"},
 typing.Optional[list[int]] = None)- **token_ids_0** (list[int]) --
```

- `**token_ids_1** (list[int], *optional*) --`

Optional second list of IDs for sequence pairs.0 `list[int]` List of zeros.

Create a mask from the two sequences passed to be used in a sequence-pair classification task. T5 does not make

use of token type ids, therefore a list of zeros is returned.

---

`class` `<span class="code-`

`inline">transformers.T5Modeltransformers.T5Modelhttps</span>`

---

`class`

```
transformers.T5Modeltransformers.T5Modelhttps://github.com/huggingface/transformers
"config", "val": "list", {"name": "T5Config"}]- **config** ([T5Config])
(/docs/transformers/main/en/model_doc/t5# transformers.T5Config)) --
```

Model configuration class with all the parameters of the model. Initializing with a config file does not

load the weights associated with the model, only the configuration. Check out the

```
[from_pretrained ()]
```

(/docs/transformers/main/en/main\_classes/model# transformers.PreTrainedModel) method to load the model weights.

The bare T5 Model outputting raw hidden-states without any specific head on top.

This model inherits from [PreTrainedModel]

(/docs/transformers/main/en/main\_classes/model# transformers.PreTrainedModel)

Check the superclass documentation for the generic methods the

library implements for all its model (such as downloading or saving, resizing the input embeddings, pruning heads)

This model is also a PyTorch [torch.nn.Module] (<https://pytorch.org/docs/stable/nn.html#torch.nn.Module>) subclass.

Use it as a regular PyTorch Module and refer to the PyTorch documentation for all matter related to general usage

forwardtransformers.T5Model.forward<https://github.com/huggingface/transformers>  
"input\_ids", "val": ": typing.Optional[torch.LongTensor] = None"}, {"name":  
"attention\_mask", "val": ": typing.Optional[torch.FloatTensor] = None"}, {"name":  
"decoder\_input\_ids", "val": ": typing.Optional[torch.LongTensor] = None"},

```

{"name": "decoder_attention_mask", "val": ": typing.Optional[torch.BoolTensor] =
None"}, {"name": "encoder_outputs", "val": ":
typing.Optional[tuple[tuple[torch.FloatTensor]]] = None"}, {"name":
"past_key_values", "val": ": typing.Optional[transformers.cache_utils.Cache]
= None"}, {"name": "inputs_embeds", "val": ": typing.Optional[torch.Tensor] =
None"}, {"name": "decoder_inputs_embeds", "val": ":
typing.Optional[torch.Tensor] = None"}, {"name": "use_cache", "val": ":
typing.Optional[bool] = None"}, {"name": "output_attentions", "val": ":
typing.Optional[bool] = None"}, {"name": "output_hidden_states", "val": ":
typing.Optional[bool] = None"}, {"name": "return_dict", "val": ":
typing.Optional[bool] = None"}, {"name": "cache_position", "val": ":
typing.Optional[torch.LongTensor] = None"}- **input_ids** (torch.LongTensor
of shape (batch_size, sequence_length)) --

```

---

**forward** transformers.T5Model.forward <https://github.com/huggingface/transformers>

---